Efficient and Proof-of-Useful-Work Friendly Local-Search for Distributed Consensus

Matthias Fitzi
IOG, Switzerland
matthias.fitzi@iohk.io

Aggelos Kiayias
University of Edinburgh & IOG, UK
akiayias@inf.ed.ac.uk

Laurent Michel
University of Connecticut
laurent.michel@uconn.edu

Giorgos Panagiotakos
IOG, Greece
giorgos.panagiotakos@iohk.io

Alexander Russell
University of Connecticut & IOG, USA
acr@cse.uconn.edu

November 13, 2025

Abstract

Blockchain protocols based on the popular "Proof-of-Work" mechanism yield public transaction ledgers maintained by a group of distributed participants who solve computationally hard puzzles to earn the right to add a block. The success and widespread adoption of this mechanism has led to staggering energy consumption devoted to solving such (otherwise) "useless" puzzles. While the environmental impacts of the framework have been widely criticized, this has been the dominant distributed ledger paradigm for years.

The Ofelimos "Proof-of-Useful-Work" protocol *(Fitzi et al., CRYPTO 2022)* addressed this by establishing that useful combinatorial problems could replace the conventional hashing puzzles, yielding a provably secure blockchain that meaningfully utilizes the computational work that underlies the protocol. The usefulness to wastefulness ratio of Ofelimos hinges on the properties of its underlying generic distributed local-search algorithm—Doubly Parallel Local Search (DPLS). We observe that this search procedure is particularly wasteful when exploring steep regions of the solution space.

To address this issue, we introduce Frequently Rerandomized Local Search (FRLS), a new generic distributed local search algorithm that we show to be consistent with the Ofelimos architecture. While this algorithm retains ledger security, we show that it also provides compelling performance on benchmark problems arising in practice: Concretely, state-of-art local-search algorithms for cumulative scheduling and warehouse location can be directly adapted to FRLS and we experimentally demonstrate the efficiency of the resulting algorithms.

Contents

1 Introduction

Blockchains are distributed ledger protocols that rely on cryptographic techniques to achieve consensus and so guarantee the integrity of ledger updates. The technology emerged over a decade ago and blossomed rapidly, playing central roles in distributed finance, cryptocurrencies, smart contracts, and other distributed-computing challenges such as supply-chain management. Bitcoin is the emblematic example of a blockchain based on Proof-of-Work as the core cryptographic building block. While the Proof-of-Work mechanism is very effective, it has staggering sustainability penalties and social costs: the protocol currently consumes approximately 18.84 GW, or 0.74% of worldwide electricity consumption [9]. The geographic distribution of Bitcoin mining further compounds these effects, as over half of these energy demands arise from especially polluting electricity sources: coal, gas, and oil. Associated annual greenhouse gas emissions in 2023 rose to a shocking 83 $MtCO_2e$.

This induced the idea of *Proof-of-Useful-Work (PoUW)*, a greener alternative where the resource outlays serve the dual purpose of implementing a secure blockchain while solving hard problems whose solutions deliver intrinsic value. While this seems like a natural idea, preserving the critical proof-of-work security guarantees while enabling more general computation was a significant challenge: early approaches either focused on specific computations of dubious usefulness (e.g., finding primes), or did not provide rigorous security conclusions. Ofelimos [19] was the first PoUW blockchain protocol to solve general, large-scale optimization problems while giving provable cryptographic security guarantees. Natural targets for such a service include hard problems that must be repeatedly solved, such as security-constrained economic dispatch (for power networks), crew scheduling (for airlines), and vehicle routing (for transportation), as well as other difficult high-value challenges such as warehouse location. In this context, the Bitcoin network can be an unparalleled computational resource if it can be harnessed: even 1% of the current mining capacity corresponds to millions of petaflops.

While [19]'s result was definitive on the security front, its usefulness hinged on a new local search algorithm whose competitiveness (and hence usefulness) was not analyzed vis-à-vis the state of the art in generic optimization. Indeed, our experimental results reveal a significant shortcoming of their optimization engine, which motivates us to develop a new approach.

A powerful, general technique for solving optimization problems is Constraint-Based Local Search (CBLS) [51], where one formulates declarative models and uses a generic solver to obtain high-quality solutions; the technique is particularly relevant for *hard and large* instances that are infeasible for complete methods and is thus an attractive foundation for a useful-work platform. Given this, *the purpose of our paper is to revisit Ofelimos and its algorithmic core in order to deliver seamless, efficient optimization of CBLS models and thus solve hard combinatorial problems with manifest intrinsic value. Moreover, we aim to establish clear evidence of efficiency, demonstrating low overhead as one transitions from a native implementation to the blockchain optimization engine.*

Towards this end, in this work we articulate how to unify Ofelimos and CBLS, evaluate how well the new architecture performs in the face of the emerging security/performance trade-offs, and identify the essential requirements one needs to consider in order to adopt other solvers and technologies for the same purpose. The remainder of this section reviews key concepts from the blockchain literature before articulating the specific contributions of the paper.

Blockchain protocols. A permissionless blockchain (e.g., as implemented in Bitcoin [43]) maintains a public transaction ledger via sequential updates without relying on a trusted third party. Instead, participating parties ("nodes") actively maintain the ledger by "mining" ledger updates in the form of new transaction-bearing blocks to be added to an ever-growing blockchain.

In general, the right to add a new block in such permissionless systems is determined by a "public lottery" that scales with possession of a somewhat scarce resource. In the case of *Proof of Work (PoW)* this

resource is *computing power*. Each miner individually—and continuously—attempts to solve a moderately hard puzzle based on its proposed block (and the current state of the blockchain). Once a miner has found a solution, it may publish its new block to extend the blockchain—we say that the miner *has found a block*. PoW blockchains like Bitcoin can be shown to be secure as long as a majority of the computing power dedicated to the system is controlled by honest parties.

Although this puzzle-solving effort is obviously useful to secure the ledger, it is not useful beyond this particular purpose. Considering the prodigious computational power currently invested in PoW blockchains, it is an important question whether this work can be targeted to solve useful real-life problems; this concept is aptly called a *Proof-of-Useful-Work (PoUW)* mechanism [4, 16].

PoUW blockchains. The security proofs of traditional Proof-of-Work mechanisms rely on particular structural properties of "hash-based" puzzles: among other features, solutions to these puzzles can be checked quickly and can be optimally discovered by a generic "guess and check" algorithm. The task of designing a secure Proof-of-Useful-Work mechanism thus requires some delicate work to ensure that analogous properties—which of course do not exist in typical optimization problems—are somehow guaranteed.

Ofelimos [19] is a PoUW blockchain protocol proven secure in a rigorous, cryptographic sense. The useful work performed by the protocol consists of running a distributed stochastic local-search (SLS) algorithm to solve optimization problems submitted by interested clients. From a blockchain perspective, Ofelimos essentially runs a variant of Bitcoin wherein the PoW is replaced by PoUW.

In Doubly Parallel Local Search (DPLS), the generic distributed local search algorithm realized by Ofelimos, each miner runs a "classical" fixed-step local search algorithm a number of times, each time starting from the same point. The best endpoint is then shared in the network and other miners may decide to use it as the starting point of their search.

1.1 Our contributions

(I) Wastefulness of DPLS. We observe that the search approach of DPLS has an inherent weakness: running multiple local searches that start from the same point is wasteful if the initial point resides on a steep slope—running local search just once would result in arriving at roughly the same point as picking the best result out of multiple runs. This weakness is translated in potentially huge amounts of work/energy wasted in non-useful/repeated work in case of systems of the size of Bitcoin. We substantiate our claims further by providing experimental results showcasing the performance loss of DPLS compared to classical local search.

(II) Frequently Rerandomized Local Search (FRLS). We introduce FRLS, a new algorithmic paradigm for Ofelimos. In FRLS, a miner iterates the following high-level tasks: First, it partially rerandomizes its current solution candidate by slightly perturbing the respective solution parameters, and then runs local search for a fixed number of steps, starting from the perturbed candidate, to produce a new candidate solution.

The miner in FRLS does not run multiple walks starting from the same point, and thus the performance weakness of DPLS is avoided. Moreover, the rerandomization step essentially smooths the *hardness* of the computation, with the effect of avoiding attacks where a malicious party predicts the local search output without running the full computation, thereby gaining an advantage in finding new blocks and thus degrading blockchain security.

Interestingly, randomization in FRLS serves an unusual dual purpose. As common in SLS, randomization of decisions is used to make the computation *reasonably easier* by helping to escape from local minima (as a form of diversification). However, the second purpose is contrary: it is used to guarantee that the

local-search steps are *sufficiently hard* to compute, preventing adversaries from exploiting predictable characteristics of a given starting point. This dual nature creates a tension between the security and optimization objectives; yet, in practical settings (shown through an ablation study) the extended use of rerandomization is manageable and, at times, even beneficial.

(III) Real-world instantiations. We showcase the generality of FRLS by adapting two established state-of-the-art algorithms—one for cumulative scheduling [41] and one for uncapacitated warehouse location [40]—to our framework. The "translation" from the classical model to FRLS is straightforward and the essential structural properties of the two algorithms are preserved in the process. We then empirically evaluate their performance on well-known benchmarks, comparing their running times against the original algorithms. The results clearly show that the adapted algorithms' performance is on par with the original ones. Furthermore, they show that a parsimonious use of additional randomization strengthens the security properties, yet has a limited and controlled impact on the solution quality.

1.2 Related work

Scheduling is a vibrant research area with dozens of variants studied during the past sixty years. Surveys [25, 26, 42] give a sense of both breadth and liveliness. While Constraint Programming and Mixed Integer Programming are often top contenders, the Resource Constrained Project Scheduling Problem (i.e., all RCPSPs are Cumulative Scheduling Problems) remain computationally very challenging and Iterative Flattening Search (IFS) [11, 41, 47] still dominates this group. Accelerating the resolution of RCPSPs instances with ML is possible [31, 38, 55] and compatible with this framework with one caveat. *The computational state maintained by Local Search algorithms is concise and thus suitable for inclusion on the blockchain*. The same *does not hold* for deep neural networks models (with many millions of weights) typical of modern machine learning to prove completion of work and disseminate these results to peers is infeasible with current on chain storage capacities. Addressing this challenge is an open question. In contrast, local search algorithms, even when applied to difficult problems of interest, are quite succinct. We remark the datasets used to train learning models are also large; however, this is presumably less of a difficulty: as training data is static it is reasonable to expect that a convention that commits to these data at the outset and indexes those portions that are used for particular training phase might be feasible. Finally, in the context of the particular problems discussed in this article, we note that local search is the state-of-the-art (and, in particular, outperforms known machine learning frameworks).

Early approaches to PoUW involved the computation of large prime numbers [21, 30], yet without giving any concrete security guarantees. More commonly, PoUW was devised to perform stochastic local search [16, 39, 48], or, machine learning [3, 12, 13, 36, 37, 50, 52, 54]. While some of the mentioned work considers security aspects like robustness against malicious updates or privacy of the training data, the security of the resulting consensus protocol is consistently either neglected or defended by superficial arguments.

The first provably secure construction for PoUW was given in [4], however it is not fit for blockchain consensus as, for example, it does not introduce any variance to puzzle-completion time. Blockchain-compatible PoUW with provable security guarantees was first demonstrated in [29], where the useful work consists of computing SNARKs [6, 45], a variant of cryptographic zero-knowledge proofs. Using some techniques from Ofelimos, [49] gave a provably secure construction for PoUW consensus that performs deep learning as the underlying PoUW computation—though assuming a weaker adversary than Ofelimos. As with any proposal to leverage computational problems with large inputs for PoW consensus (e.g., the training dataset for the model), this approach introduces a new data-availability challenge: the input data cannot be realistically maintained on chain, but must have guaranteed availability in order to verify PoW certificates.

Organization of the paper. In Section 2 we review the Ofelimos protocol and CBLS models of cumulative scheduling and uncapacitated warehouse location. The wastefulness issue of DPLS as well as the new FRLS paradigm are introduced in Section 3. In Section 4, two "real-world" instantiations of FRLS are presented. Experimental results are presented in Section 5.

2 Preliminaries

2.1 Ofelimos

In Bitcoin [43], PoW is implemented by repeated hashing: a counter inside the proposed block is increased until the hash value of the block lies below a given threshold—constituting the event of finding a block. Ofelimos essentially runs a variant of Bitcoin wherein this PoW is replaced by PoUW. For our purposes, the details of how the useful work is embedded in the PoUW function are not crucial, as in Ofelimos the useful-work component can be essentially treated as a black box. For completeness, we still summarize the PoUW functionality of Ofelimos—and give a comprehensive description in Appendix A. To motivate the architecture, we first reiterate some issues the design needs to resolve.

Issue 1: Shortcuts. A miner must not be able to reduce the time complexity of its useful-work computations by biasing the randomization towards easy-to-compute instances, and thus gaining an advantage in the block lotteries over miners that apply uniform randomness.

Issue 2: Verification overhead. The protocol must be secured against adversarial behavior. To that end, all nodes in the network need to verify that the block miners performed their PoUWs correctly. The ratio of verification computation in the network per executed useful-work computation must be minimized in favor of the overall "usefulness" of the PoUW blockchain.

The (generic) PoUW function of Ofelimos, motivated by the above issues, is depicted in Fig. 1. It consists of the three main stages *pre-hashing*, *useful-work*, and *post-hashing*.

```
1   PoUW(block, problem,algSt, localSt ): ⟨blockFound,cnt,algSt,localSt⟩ {
2       cnt = 0;
3       do { // pre−hash
4           cnt = cnt + 1;
5           preH = Hash(block,problem, algSt , cnt );
6       } until ( preH < T1 );
7       seed = Hash(preH); // (pseudo)random
8       ⟨algSt',localSt'⟩ = usefulWork(problem,algSt , localSt ,seed);
9       postH = Hash(preH, algSt');
10      blockFound = postH < T2; // post−hash
11      return ⟨blockFound,cnt,algSt',localSt'⟩;
12  }
```

Figure 1: The pseudo-code for a single PoUW attempt.

Pre-hashing. In the pre-hashing stage (lines 3-7), the miner solves a single instance of PoW (à la Bitcoin) to obtain a seed for the useful-work computation to follow. The average-case complexity of this PoW computation is set at least as high as the worst-case complexity of the useful-work computation.As a byproduct,

on line 8, the PoW computation yields an (unpredictable) `seed` for the (pseudo-)randomness used by the useful computation.

It is important to note that, due to the pre-hashing PoW being as hard as the useful computation, at most half of the overall work performed by miners is directed to solving useful problems. Yet, as stated in Section 1, even 1% of the current work directed to Bitcoin corresponds to millions of petaflops—and is well worthwhile to be replaced by useful computation.

Pre-hashing prevents Issue 1: to obtain a (valid) seed for the useful-work step, in expectation, the adversary needs to spend at least as much computation as for the useful-work itself. Given a seed that defines a useful-work instance with above-average hardness, grinding (i.e., re-executing the pre-hashing stage) for another instance (i.e., seed) hence does not offer faster useful-work completion over computing the hard instance itself.

Useful work. On line 9, the useful computation is executed. The computation takes as input (i) the description `problem` of the "useful" optimization problem to be solved, (ii) the state (`algSt` and `localSt`) from which the computation should continue its operation, and (iii) the `seed`. Note that state is split in two parts: `algSt` that corresponds to the part of the state that will end up being posted in the blockchain, and `localSt` that is only stored locally by the miner. The updated state is output by `usefulWork`.

It is important to note that the useful-work part consists of a *fixed* number of computation steps (using a given random seed) of a predefined *fixed* algorithm. Thus, even if a faster solver were to become available for this task, a miner could not use it to speed up their PoUW computation to gain an advantage over the other miners.

Post-hashing. In the post-hashing stage (lines 10-11), `preH` together with `algSt'` are hashed against a threshold to determine whether a new block has been found—along the lines of how Bitcoin determines whether a block can be published. In case `postH` is smaller than `T2`, the miner publishes `block` together with information about a *constant* number of `usefulWork` runs.[1] Note that, whether or not a block can be published does not depend on the quality of the state computed in the useful-work step.

Post-hashing addresses Issue 2: it guarantees that, in expectation, a large number of useful-work instances need to be solved until one of them is eligible for publication. Of those many instances, only the published ones must be publicly verified.

Requirements. The given PoUW construction allows for quite a broad class of `usefulWork` instantiations while retaining the provable security guarantees of Ofelimos. This paves the way for using Ofelimos with alternative optimization techniques.[2] The main requirements from `usefulWork`, extracted from [19], are listed next:

1. `usefulWork` must be moderately hard. Reason: a malicious miner must not be able to dramatically speed up the computation of one or more invocations of `usefulWork` when the (pseudo-)randomness of the computation is selected in a uniform manner.
2. `problem` and `algSt` must be relatively small—in the order of KB. Reason: the blockchain is the only reliable data source. Thus the (initial) problem specification and the respective state updates must fit in a block, and current block sizes in mainstream blockchains (Ethereum, etc.) are in the order of KB.

[1] Besides the `usefulWork` run that led to the small `postH` which is *always* published to allow for the verification of PoUW, any other run published is determined by the specific optimization protocol implemented by Ofelimos. An obvious choice is to publish the run that led to the best solution.

[2] Examples will be shown in later sections when looking at two distinct instantiations of the `usefulWork` procedure based on cumulative scheduling and uncapacitated warehouse location.

3. `problem`, `algSt`, and `seed` should fully determine the output of `usefulWork`. Reason: the only degree of freedom to advance `algSt` must be the `seed` which is an unpredictable result of pre-hashing. Otherwise, pre-hashing could be bypassed by the adversary.
4. Running `usefulWork` with `localSt` available should be in the order of seconds. Reason: (relatively) fast computation of a PoW solution attempt is required in favor of good blockchain transaction latency and frequent state updates towards solving the useful-work problem.
5. Verifying the correctness of a run of `usefulWork` without `localSt` available should be in the order of seconds. Reason: (relatively) fast verification in comparison to computing a PoW attempt is required for blockchain security.

2.2 Cumulative Scheduling

Major industries (e.g., transportation, airlines, manufacturing) must contend, daily, with scheduling problems despite their inherent computational hardness. Disjunctive and cumulative scheduling (a.k.a., resource constrained project scheduling) are among the most studied and remain challenging, despite 6 decades of active research [26, 53].

Cumulative scheduling [5] aims to minimize the project duration, i.e., the makespan, subject to both precedence constraints between non-preemptible activities as well as finite discrete resource constraints required to execute tasks. Resolution techniques include Integer Programming [42], Boolean Satisfiability and their Satisfiability Modulo Theory brethren [8] as well as heuristic –incomplete– methods [7, 10, 11, 41, 44] or evolutionary algorithms such as Ant Colony Optimization [23]. Incomplete methods are attractive for large instances that are otherwise not tractable.

IFLAT-IRELAX [41] is a local search that operates on a relaxation and alternates two phases. The IFLAT phase adds machine precedences to eliminate resource *over-usage*. It ends with a resource-feasible schedule whose makespan is driven by the release dates of activities. The IRELAX phase is devoted to relaxing machine precedences in order to eliminate resource *under-usage* by exposing and removing nested bottlenecks.

2.3 Uncapacitated Warehouse Location

Infrastructure planning requires solving facility location problems, i.e., locating a to-be-determined number of locations for shipment facilities for any given commodity. Specifically, given a set of n warehouses sites and m clients, the problem focuses on choosing at which of the n sites to build warehouses (with infinite capacity) to serve the m clients at the minimal cost.

Costs are incurred to open a warehouse at a site and to transport goods from sites to clients. Clients are always to be served from the closest open warehouse to minimize transportation costs. The problem was studied extensively [17, 20, 24] using global methods. It was also tackled with simulated annealing, Tabu search and genetic algorithms [1, 2, 32–34] as well as a CBLS method [40] with the express purpose to scale to large instances with thousands of customers and clients.

3 Revisiting Local Search in Ofelimos

Ofelimos provides a generic platform for performing useful computations of interest by adequately instantiating the `usefulWork` function (Fig. 1). In context of distributed local search, the paper describes a paradigm for implementing such algorithms: Doubly Parallel Local Search (DPLS). We now revisit DPLS and argue that it suffers from some intrinsic inefficiencies. Then, we introduce Frequently Rerandomized Local Search (FRLS), a new and vastly more efficient paradigm for instantiating `usefulWork` over Ofelimos.

3.1 DPLS and its inefficiencies

DPLS calls for miners to repeatedly run local search for a fixed number of steps, starting from points in the solution space found by other miners. In more detail, a set of random starting points is initially available to the miner. It selects one of them and, during each invocation of `usefulWork`, produces a *path update*, i.e., it runs the specified local search algorithm for a fixed number of steps starting from the point selected. This process is repeated—always starting from the same initial point (see Fig. 2)—until a new block is found. When that happens, the miner publishes the *best* path update observed so far and picks a new point from the ones available on the blockchain to continue working. Following this, other miners may adopt the endpoint of the newly published path.

An adaptation of WalkSAT for DPLS is presented in [19] as a proof of concept. In the adaptation, starting points are WalkSAT configurations and path updates correspond to running WalkSAT for a fixed number of steps. The performance of the adapted algorithm was experimentally shown to be comparable with that of WalkSAT [Fig. 6, [18]] when partially solving instances of the Automated Planning problem [22]. Yet, performance is not that tight and it deteriorates as the number of path updates per block increases [Fig. 5, [18]], thus limiting the effectiveness of the algorithm.

As explained next, this inefficiency is not accidental, but rather an undesirable characteristic of the paradigm. As the miner explores different places in a non-trivial solution landscape, it will often choose a starting point that resides on a steep slope. In this case, it would be sufficient to produce a single path update to get to the bottom of the slope. Instead, in DPLS the miner *always* computes multiple path updates starting from the same point. Yet, the best of them is going to end up at approximately the same place as a single invocation, implying that the extra paths computed were wasteful.

We empirically validate our above claims in the next subsection. First, we present the FRLS paradigm, a different approach to run local-search over Ofelimos that better uses available resources.

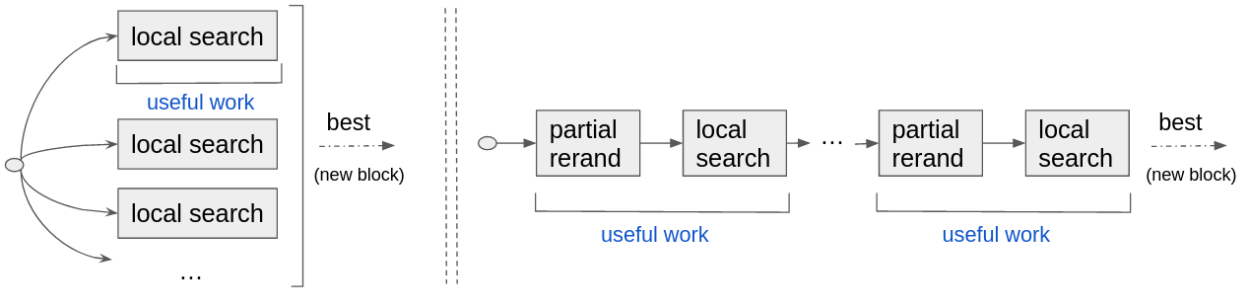

Figure 2: Useful work performed by a single miner searching for a new block in DPLS (left)/ FRLS (right); pre/post-hash steps not shown.

3.2 Frequently Rerandomized Local Search

To overcome the limitations of DPLS we take a different approach, closer in spirit to local search algorithms found in the literature. Frequently Rerandomized Local Search (FRLS), our new paradigm, has each miner produce a sequence of path updates. In contrast to DPLS where each path update starts from the same point, in FRLS, the end point of each path update in the sequence serves as the starting point of the next path update. Whenever a miner finds a new block, it outputs the *best* path update it has observed since its last published block, and continues its search uninterrupted.

Essentially, different miners run independent local searches in parallel, a typical strategy when running local search in a distributed manner [15]. This is especially important in the context of blockchain protocols,

where coordination only happens periodically through publishing new blocks, typically every few seconds. As these systems allow for massive parallelization (in the order of thousands), coordination among miners may be prohibitively costly or slow. Thus, FRLS is ideal in this sense, as it does not require miners to share any algorithmic state to make progress.[3]

If, in addition, path updates consisted only of a fixed-step local search invocation, as in DPLS, then the approach outlined above would be no different than running a classical local search algorithm. However, such a choice is problematic from a security perspective due to the hardness requirement of `usefulWork` mentioned on Section 2. Namely, for performance reasons, miners do not publish all the path updates computed towards finding a block. Hence, a malicious miner may choose an arbitrary starting point for its search that it has pre-analyzed and where predicting the result of the path update may be computationally easy. This would downgrade the security guarantees of Ofelimos, as the malicious party would be able to "multiply" its power to influence the protocol by computing PoUWs faster. Note that this is not a problem in DPLS, since the starting point of the search must be drawn from those already published in the blockchain.

Ideally, from a computational-hardness perspective, to avoid precomputation attacks we would like the miner to start each path update computation from a random point. However, this would be catastrophic from an optimization perspective, as any work done previously would be lost. Instead, we take a more granular approach. Before running the fixed-step local search part of each path update, we have miners partially rerandomize the starting point coming from the previous path update, see Fig. 2. This way, progress is not lost and the starting point of the search cannot be completely controlled; recall that controlling the randomness of the path update is computationally expensive for the miner due to pre-hashing.

Example 1 (WalkSAT revisited) *We briefly outline an FRLS variant of WalkSAT. Search starts with the miner selecting a random starting configuration C_0. Then it first rerandomizes a number of variables of C_0 to obtain C_0', and then runs classical WalkSAT starting from C_0' for a fixed number of steps to obtain C_1. The process is repeated to obtain in turn $C_1', C_2, C_2', \ldots$, until the blockchain protocol instructs the miner to stop working on this instance. Meanwhile, whenever the miner produces a block, it publishes the best configuration observed since its previous block production event.*

As promised, next we empirically validate the observations made in Section 3.1 about the inefficiencies of DPLS-WalkSAT compared to classical WalkSAT as well as FRLS-WalkSAT. Unlike [18], instead of focusing on the time it takes to partially solve a problem instance, we fully map the rate at which the different algorithms converge to better solutions (towards minimizing the number of false clauses). We focus on the same Automated Planning instances[4] used in [18], and examine the performance of a single search thread running repeatedly the three algorithms outlined above; in the case of DPLS-WalkSAT the best endpoint produced after a fixed number of path updates all starting from the same point is selected to continue the search.[5] A representative sample of the results is presented in Fig. 3, where we observe that while classical WalkSAT and FRLS-WalkSAT quickly converge to a good solution, DPLS-WalkSAT takes a while to converge to solutions of similar quality. The same results are observed in the full set of experiments performed, presented in Appendix B.

The requirements for `usefulWork` (Section 2) mandate that path updates take a few seconds to be computed. This implies that the rerandomization step should be repeated as frequently. This modification of the algorithm departs from standard local-search restarting or diversification strategies, where rerandomization is cued by current features of the search, e.g., getting stuck on a local minima, and thus is not thoroughly explored in the local-search literature. As we concretely show in the next sections, real-world state-of-the-art local-search algorithms can be adapted to FRLS without significantly compromising their performance.

[3] Still, for security reasons miners *do* have to receive new blocks in time.

[4] SAT-encoded instances for blocks world planning problems from the SATLIB [28].

[5] `https://anonymous.4open.science/r/FRLS-DDEC/walkSAT/` has source code and scripts.

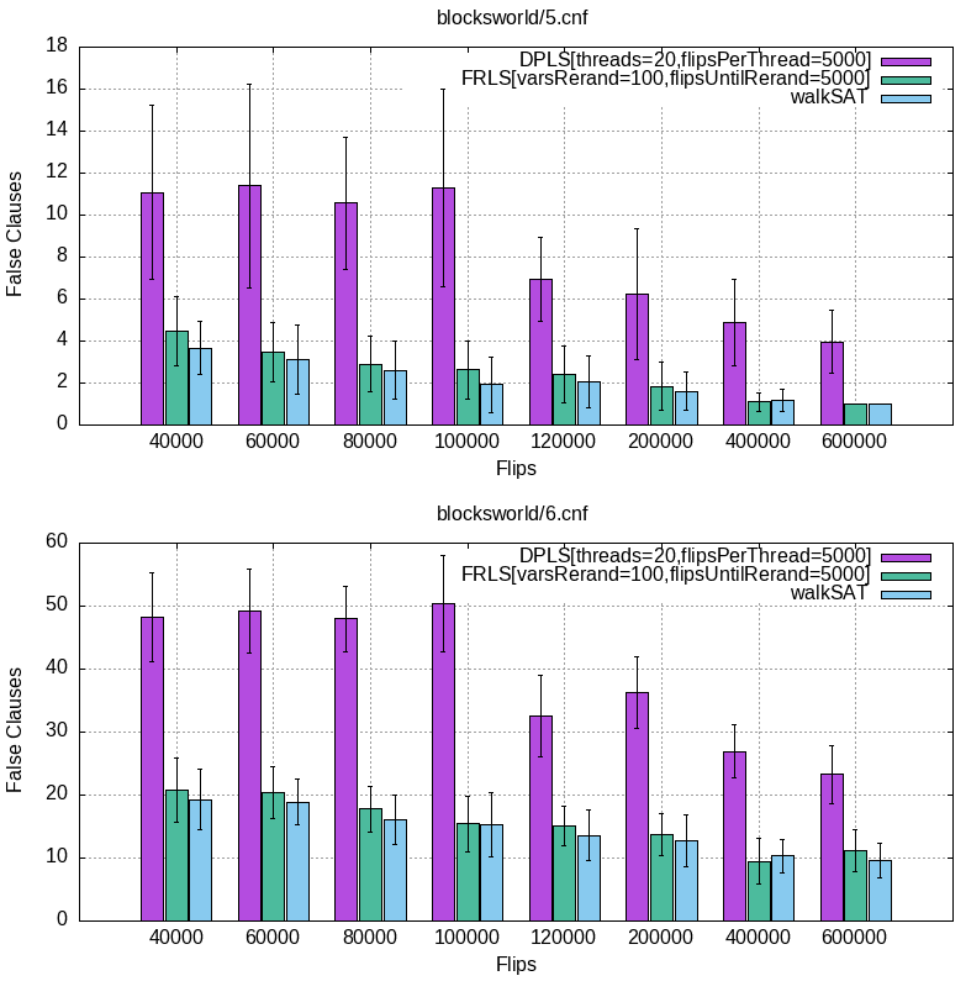

Figure 3: Comparison of classical WalkSAT, DPLS-WalkSAT (Ofelimos), and FRLS-WalkSAT. DPLS uses 20 path updates starting from the same point and each carries out 5000 flips, while FRLS rerandomizes 100 variables every 5000 steps.

Remark 1 *Besides attacks against the useful work relating to the selection of an "easy" starting point, a malicious party may try to plant a problem instance (while pretending to be a client of the system) that has many such "easy" starting points, in an effort to side-step our defenses. In principle, attacks of this kind can be dealt with in the same way, e.g., by adequately and randomly perturbing the optimization problem solved. Related ideas were explored [14, 46] to perturb local minima and neighborhood to assist local search.*

The security analysis of Ofelimos running DPLS in [19] follows the standard security methodology of reducing the security of the protocol to a well-defined computational assumptions that can be studied independently of the rest of the protocol, in that case, the "moderate hardness" of the DPLS path update function [Definition 2, Assumption 3, [19]]. Interestingly, we can reuse the reduction presented there with minor changes to reduce the security of Ofelimos running FRLS to the assumption that the FRLS path update function is "moderately hard"[6]; roughly, it must be the case that computing k invocations of the path

[6]Hardness of the DPLS path update function was motivated by the fact that the starting point of any such computation must be an output of the local search algorithm, and thus by carefully designing the path update function it can be sufficiently random. In FRLS we take a simpler, more generic, and possibly easier to study approach, and instead base hardness on the fact that the

update function must take time proportional to k. We state the resulting theorem informally here, and we point to [Corollary 1, [19]] for further details.

Theorem 1 *(Informal.) Assuming that FRLS's path updates are moderately hard and that the adversarial computational power as well as block production are sufficiently bounded, it holds that Ofelimos (where the* `usefulWork` *function is instantiated with FRLS) satisfies Persistence and Liveness.*

4 FRLS Instantiations

This section presents the FRLS adaptations of local search algorithms for cumulative scheduling and uncapacitated warehouse location.

4.1 Cumulative Scheduling

Problem specification. Consider a set of activities $A = \{a_1, \cdots, a_n\}$ with duration $d(a_i)$ and resource usage $r(a_i)$ on machine $m(a_i)$, a set of machines $M = \{m_1 \cdots, m_k\}$ with capacities $cap(m_i)$, a set of job precedence constraints $P = \{a_i \prec a_j : a_i, a_j \in A\}$, Let $\mathcal{A}(m)$ be set set of activities using machine $m \in M$ and let $H = [0..\sum_{a \in A} d(a_i)]$ be the scheduling horizon. A schedule $\sigma : A \to H$ assigns start dates from H to all activities in A and induces a makespan $mk(\sigma)$ equal to the latest finishing time. Let $O(\sigma, m, t) \subseteq A(m)$ be the set of activities on m that overlap with time t in σ. σ is feasible provided that all the constraints

$$\forall (a_i \prec a_j) \in P \quad : \quad \sigma(a_j) \geq \sigma(a_i) + d(a_i) \tag{1}$$

$$\forall t \in H, m \in M \quad : \quad \sum_{O(\sigma,m,t)} r(a) \leq cap(m) \tag{2}$$

hold. Let mk^* be the makespan of feasible *and* optimal schedule and σ^* be the set of schedules with makespan mk^*. Cumulative scheduling seeks to produce mk^* or minimize a feasible approximation of it.

Constraint-based local-search model. The IFLAT-IRELAX local search model splits the constraints into hard(1) and soft (2) constraints. Soft constraints are relaxed and the overall *violations* are minimized. A violation occurs at time $t \in H$ for machine m whenever the resource usage exceeds the capacity, i.e., when $\sum_{O(\sigma,m,t)} r(a) > cap(m)$. The set of *all* time points at which a violation occurs on machine m is thus $V(\sigma, m) = \left\{ t \in H : \sum_{O(\sigma,m,t)} r(a) > cap(m) \right\}$, and its cardinality is $nbv(\sigma, m)$. The *violation degree* at time $t \in V(\sigma, m)$ is the excess usage $vd(\sigma, m, t) = \max(0, cap(m) - \sum_{O(\sigma,m,t)} r(a))$. Given a schedule σ, a time point t and a machine m, $G(\sigma, m, t) = \left\{ O' \subseteq O(\sigma, m, t) \mid \sum_{g \in O'} r(g) > cap(m) \right\}$ denotes *guilty subsets* responsible for resource violations. The CBLS model incrementally maintains, for each soft constraint $m \in C$, its violations $V(\sigma, m)$, cardinality $nbv(\sigma, m)$, violation degree $vd(\sigma, m, t)$, and $G(\sigma, m, t)$ to efficiently retrieve activities to be sequenced.

The FRLS adaption of IFLAT-IRELAX**.** The CBLS algorithm embedded in FRLS follows [41] and is shown in Fig. 4. What sets this FRLS version apart is the `rerandomizeSol` step. It produces a *path* of local search moves, starting form an externally provided candidate solution (S, σ). The precedence

starting point of each computation is explicitly partially rerandomized, as discussed earlier. Furthermore in Section 5 we provide some initial empirical evidence that partially rerandomized path update computations related to local search plausibly satisfy the "moderate hardness" property.

graph $G(A, P \cup S)$, the release dates σ, and the supporting data-structures are the local state `localSt` maintained by the miner. The algorithm state `algSt` of the resolution published on the block chain is S: the set of machine precedences.

The maximum computational effort occuring in the loop spanning lines 10-19 is controlled through the `lim` parameter. The effort alternates the `iFlat` and `iRelax` phases as in [41]. Minimal critical subsets [35] are the smallest members of $G(\sigma, m, t)$, i.e., they cannot shed any activity and yet remain guilty. Let $MCS(\sigma, m, t) \subseteq G(\sigma, m, t)$ be the guilty subset of G with only minimal critical subsets. The `iFlat` phases sample the $MCS(\sigma, m, t)$ of machines that exhibit violations to lower *contention peaks* with the addition of machine precedences. It ends when $\forall\, m \in M \;:\; V(\sigma, m) = \emptyset$. The `iRelax` phases use several iterations (`maxRelax`) to select a subset of machine precedences on the critical path and removes them with a fixed probability. Randomized decisions embedded in both phases rely on a pseudo-random stream dictated by PoUW to limit adversarial behaviors and conveyed through `seed`. `iFlat-iRelax` outputs the last solution S it finds, as well as the best solution S^* and the local state, i.e., the precedence graph, release dates σ and the supporting incremental structures.

```
1  iFlat-iRelax(ProbDesc=⟨A, M, P⟩, algSt=S,
2         localSt=⟨G, σ⟩, seed) : ⟨⟨S, S*⟩, localSt⟩ {
3    if (S = ⊥) {
4      S = ∅;
5      G = intitializePrecedenceGraph(A, P);
6      σ = initializeReleaseDates(G);
7    }
8    ⟨S*, σ*, best, nbStable⟩ = ⟨S, σ, mk(σ), 0⟩;
9    (S, σ) = rerandomizeSol(S, σ, add, seed);
10   while(lim > 0) {
11     ⟨S, σ⟩ = iFlat(S, σ, seed);
12     if (mk(σ) < best)
13       ⟨S*, σ*, best, nbStable⟩ = ⟨S, σ, mk(σ), 0⟩;
14     else nbStable += 1;
15     if nbStable > maxStable break;
16     forall(j ∈ 1..maxRelax)
17       ⟨S, σ⟩ = iRelax(S, σ, seed);
18     lim -= 1;
19   }
20   return ⟨⟨S, S*⟩, localSt⟩
21 }
```

Figure 4: The iFLAT-iRELAX function adapted for FRLS. `lim`, `add`, `maxStable` and `maxRelax` are fixed parameters of the function.

The `rerandomizeSol` routine (called line 9), critical to the security of PoUW (see Section 3), consists of adding $\lfloor \frac{add}{|M|} \rfloor$ *legal* random machine precedences. The precedence (a, b) is legal provided that $a \neq b$ and the earliest start time of a is strictly before the earliest finish time of b to prevent the formation of cycles.

This step prevents adversaries from taking advantage of maliciously selected *easy starting points* to run iFLAT-iRELAX. It is computationally hard to rewind, as it requires the computation of a new small pre-hash and in turn a new seed, which is expected to be as costly as running the rest of the function. There is a natural tension between the desire for security (which would ask for fully randomizing the starting point) and the desire to achieve progress on the combinatorial problem (which would ask for no randomization at all). This tension is reflected in the `add` parameter whose effect is investigated in the empirical section, where we observe that one can moderately increase the `add` parameter (up to some point), and thus partially

rerandomize the starting point, without incurring a significant loss in the ability to solve the combinatorial problem.

4.2 Warehouse Location (UWLP)

Problem specification. Given a set of n warehouse sites W and a set of m clients S, the cost of building a warehouse at site w is f_w while the transportation cost from warehouses to clients is given by the matrix c_{ws}. Let $Open \subseteq W$ be a decision variable indicating which sites host warehouses. The problem is to minimize the objective function f over all possible warehouse hosting scenarios with f defined as $f = \left(\sum_{w \in Open} f_w + \sum_{s \in S} \min_{w \in Open} c_{ws}\right)$. The instance is feasible when $Open \neq \emptyset$ and the problem is an unconstrained minimization. $Open$ is a Boolean vector y of size n that indicates for each site whether it hosts a warehouse.

Constraint-based local-search model. A simple (yet effective) neighborhood for CBLS is to flip the value of Boolean y_i to open or close a warehouse at that site. The n neighbors of a configuration y are therefore $N(y) = \{\text{flip}(y, i) \mid \forall i \in W\}$, where $\text{flip}(y, i)$ is everywhere identical to y except for negating the Boolean of site $i \in W$. Given a solution y, CBLS uses the *discrete gradients* of f at y in each of the n directions to decide which neighbor from $N(y)$ to move to. A CBLS model maintains the gradients incrementally and allows to query for a warehouse site $i \in W$ the value of $\Delta_i(y) = \frac{f(y) - f(\text{flip}(y,i))}{1}$. Non-degrading moves have $\Delta_i(y) \geq 0$ and CBLS can choose the neighbor of y with the largest $\Delta_i(y)$. Details on how to maintain the gradients can be found in [40].

The FRLS adaption of the CBLS Model. The Tabu search algorithm for UWLP [40] forms the basis of the `usefulWork` sub-routine. The routine takes as input the problem description (the f_w vector and the c_{ws} matrix). It also reads `algSt`, the state of the current solution y and uses `localSt` to hold onto the incremental data-structures recommended in [40] (i.e., a collection of priority queues). The `flip` parameter controls how much rerandomization to introduce between rounds. The `lim` parameter bounds the overall number of iterations. Lines 3-6 handle initialization in case y is undefined ($\perp$). The initial entries of y are set uniformly at random (but not all False) and the collection of priority queues is setup. The body of the algorithm (lines 9-23) greedily performs the best non-degrading and non-Tabu flip.

The flipped warehouse is then marked Tabu for a fixed time duration (the `when` instruction removes i from the Tabu list at a specified iteration number). If all moves are degrading, a random warehouse is *closed*. At each iteration, `selectMax` (line 10) greedily chooses a non-Tabu warehouse that maximizes $\Delta_i(y)$. To speed up the process, one can instead be less greedy and choose an improving move via a `selectFirst` to pick the first warehouse i delivering a non-negative $\Delta_i(y)$. If no such move exists `selectFirst` behaves like `selectMax`. The algorithm returns both the last and best solutions found along the trajectory along with the local state, i.e., the collection of priority queues used for the customers. The overall algorithm is shown in Fig. 5.

Once more, the call to `rerandomizeSol` (line 8) is critical to the security of the algorithm. This rerandomization is meant to strike the right balance between security and effectiveness of the combinatorial search. In this setting, `rerandomizeSol` modifies y (the vector of warehouse's state) by selecting `flip` variables in y (uniformly at random) and inverting their truth value.

5 Experimental Results

Next, we experimentally assess the quality of solutions from FRLS compared to those of state-of-the-art optimization solvers. The goal is to demonstrate that the rerandomization steps *mandated* by the security of the

```
UWLP_TS(ProbDesc=⟨f_w, c_ws⟩, algSt=y,
       localSt=PQ, seed):⟨⟨y, y*⟩, localSt⟩{
  if (y = ⊥) {
   forall(i ∈ W) y_i = random({True, False}); \# randomly set each site
   forall(i ∈ S) PQ_i = createPQueue(i, c_*s); \# one priority queue per
       customer
  }
  ⟨y*, best, nbStable, iter⟩ = ⟨y, f(y), 0, 0⟩;
  y = rerandomizeSol(y, flip, seed);
  while (lim > 0) {
   selectMax(i ∈ W \ T)(Δ_i(y)) {
     if (Δ_i(y) ≥ 0) {
       y = flip(y, i);
       T = T ∪ {i};
       when iter = iter_now + tLen ⇒ T = T \ {i};
     } else let w=random(seed, W) in y_w = False;
     if f(y) < best
       ⟨y*, best, nbStable⟩ = ⟨y, f(y), 0⟩;
     else nbStable+ = 1;
     if nbStable > maxStable break;
     iter += 1;
   }
   lim -= 1;
  }
  return ⟨⟨y, y*⟩, PQ⟩;
}
```

Figure 5: Tabu Search adapted for FRLS. `lim`,`flip` `maxStable` are fixed parameters.

blockchain protocol do not adversely affect the quality of the solutions computed from the `usefulWork` mining process. An ablation study measures the impact of the rerandomization on solution quality and the security of the blockchain. Regarding security, we focus on the rate at which local search revisits locations already seen as a function of rerandomization (lower is better). The reason is that such a study can inform us about the predictability of the search, in turn indicating a possible security risk. Fig. 6, 7, 8, 9, 10, and 11 encode with each bar color the amount of rerandomization. The bar heights convey either solution quality or collision frequency.

The experiments use a simulated blockchain and the CBLS implementation uses the Objective-CP system [27].[7] Each benchmark was run 20 times on an 2.10GHz Intel Core i3-8145U under Linux. The pseudo-random number generator's seed is parametric and set to fixed values for reproducibility. The next paragraphs discuss the results for Cumulative Scheduling and Warehouse Location.

Cumulative Scheduling. Fig. 6 uses the Lawrence LA38 (Fig. 7 provides results for the LA24 instance). Activities are replicated 3 times and the resources have a discrete capacity of 3. The left chart reports on the solution quality one can secure for different numbers of steps (conveyed here as a limit on the runtime); pre and post-hash steps are ignored in the experiment, as their impact on runtime is entirely predictable, effectively doubling the runtime as explained in Section 2. Each vertical bar shows the solution quality (along with error bars at the top to show the variability over different runs) for different settings of one parameter: The number of arcs to be added at each `rerandomizeSol` invocation which occurs every

[7] `https://anonymous.4open.science/r/FRLS-DDEC/` has the source code and scripts.

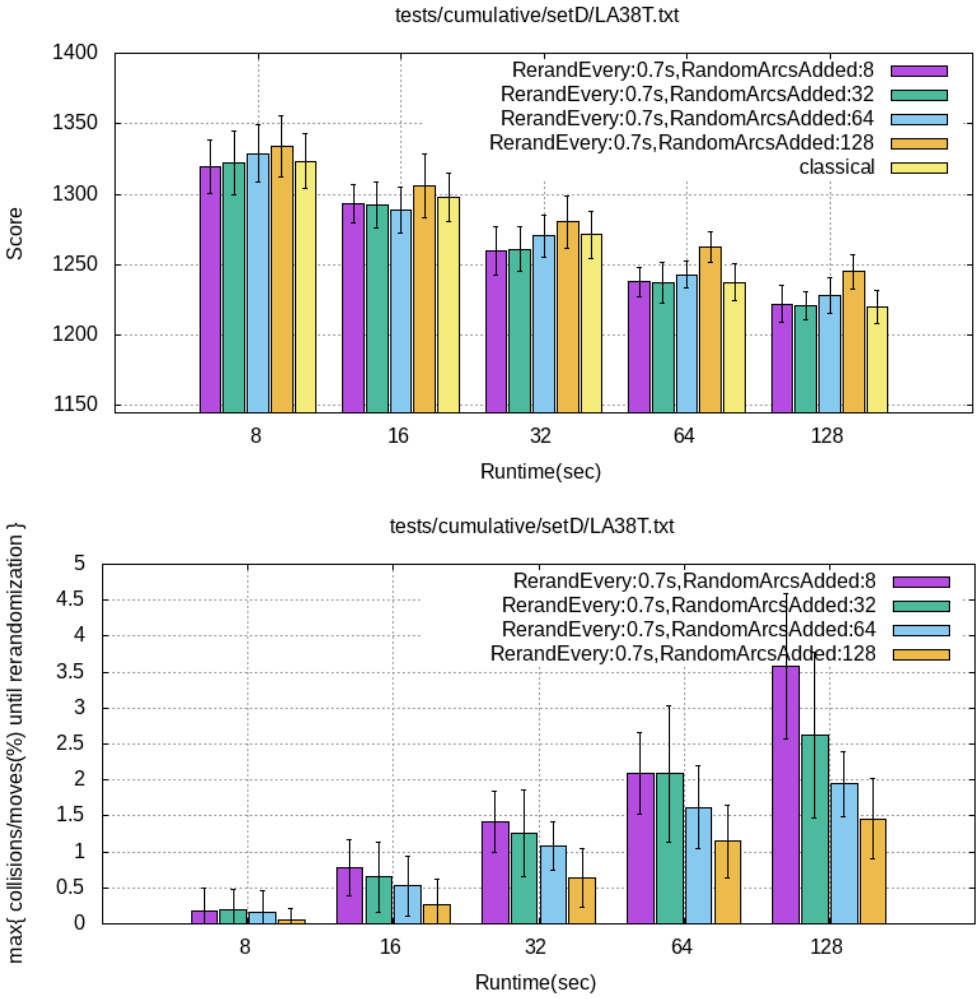

Figure 6: Cumulative Scheduling: LA38 Triplicate. Solution Quality (left) and Collisions (right).

0.7 seconds. The "classical" bar reports the results from IFLATIRELAX, running on the same hardware. Increasing the amount of rerandomization worsens the solution quality no matter the total amount of time. Yet, without an excessive amount of rerandomization (< 128 arcs), doubling the runtime allows to recover the losses induced by the rerandomization. For shorter runs though, the more aggressive rerandomization can *sometimes* improve quality.

The right chart offers a measurement of the maximum percentage of *point collisions* between any two consecutive rerandomization steps: a collision occurs when local search revisits a location explored earlier. This experiment is a sanity check for `usefulWork` hardness, as a large collision rate would imply a tendency to revisit points the algorithm has already seen, and thus its trajectory may be somewhat predictable. Output predictability of `usefulWork` could in turn be used by a malicious miner to cheaply compute blocks and degrade the security of the blockchain protocol. As expected, the rerandomization effectively counteracts this propensity and can keep the percentage of revisited states below 2% with a rerandomization adding 64 arcs (at the 128s time limit).

Fig. 7 describe the results for the `LA24` instance in triplicate (namely, every activity, along with its dependencies, was repeated 3 times). The results echo what was transpired for the `LA38` instance.

Figures 8 and 9 are similarly structured and focus on the classic `MT10` and `MT20` benchmark, also

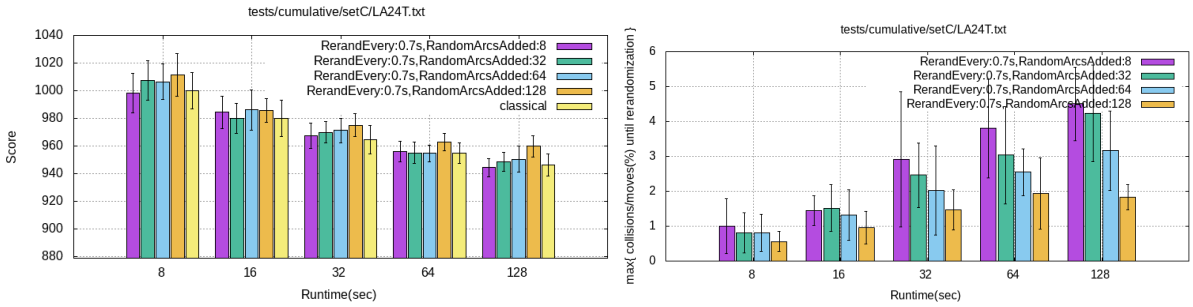

Figure 7: Cumulative Scheduling: LA24 Triplicate. Solution Quality (left) and Collisions (right).

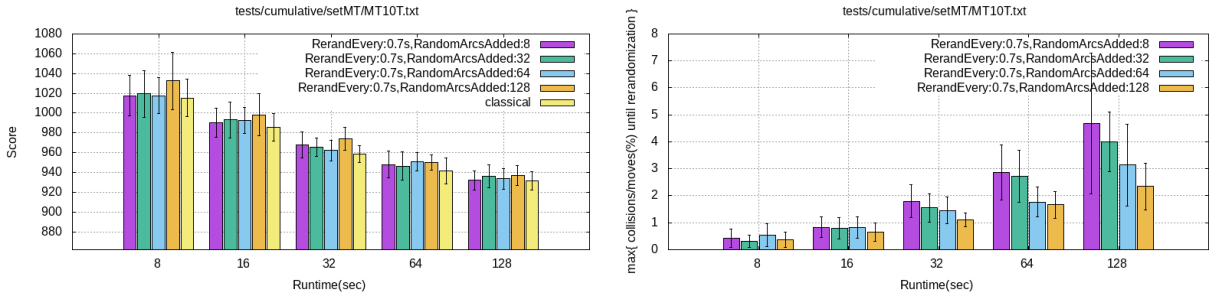

Figure 8: Cumulative Scheduling: MT10 Triplicate. Solution Quality (left) and Collisions (right).

in triplicate. The rerandomization is not harmful to quality: even the highest levels are suitable with little impact across the board that stays within the error bars. The right chart shows that rerandomization reduces state repetitions effectively.

Warehouse Location. Two large instances from [34] are evaluated that are well beyond the reach of exact mathematical programming methods, `MS1` (1000 warehouses and 1000 stores), and respectively `MT1` (2000x2000). The quality for `MS1` no longer improves much past 24 seconds and the right panel of Fig. 10 shows that an increase in runtime induces an increase in collisions that one can mediate with a rerandomization change from 15 to 25.

`MT1` (Fig. 11) conveys a slightly different message. It shows that large amounts of early rerandomization can improve the quality (time budget ≤ 64 seconds) and that, with more time, behavior similar to Cumulative Scheduling re-emerges with the rerandomization slowing down the convergence. This is likely due to rerandomization destroying (at a rate that scales with the number of flips) local structures that are delicate

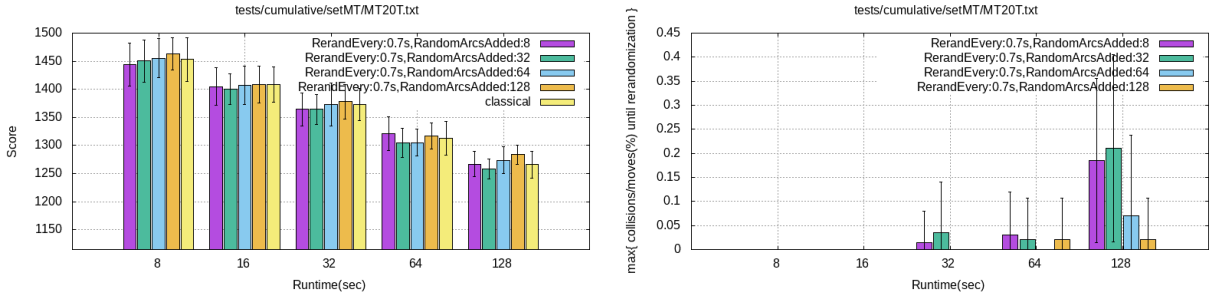

Figure 9: Cumulative Scheduling: MT20 Triplicate. Solution Quality (left) and Collisions (right).

to recover. On the other hand, the right panel of Fig. 11 shows that, even at very low rerandomization, the search space is so vast that the algorithm does not experience *any* collisions.

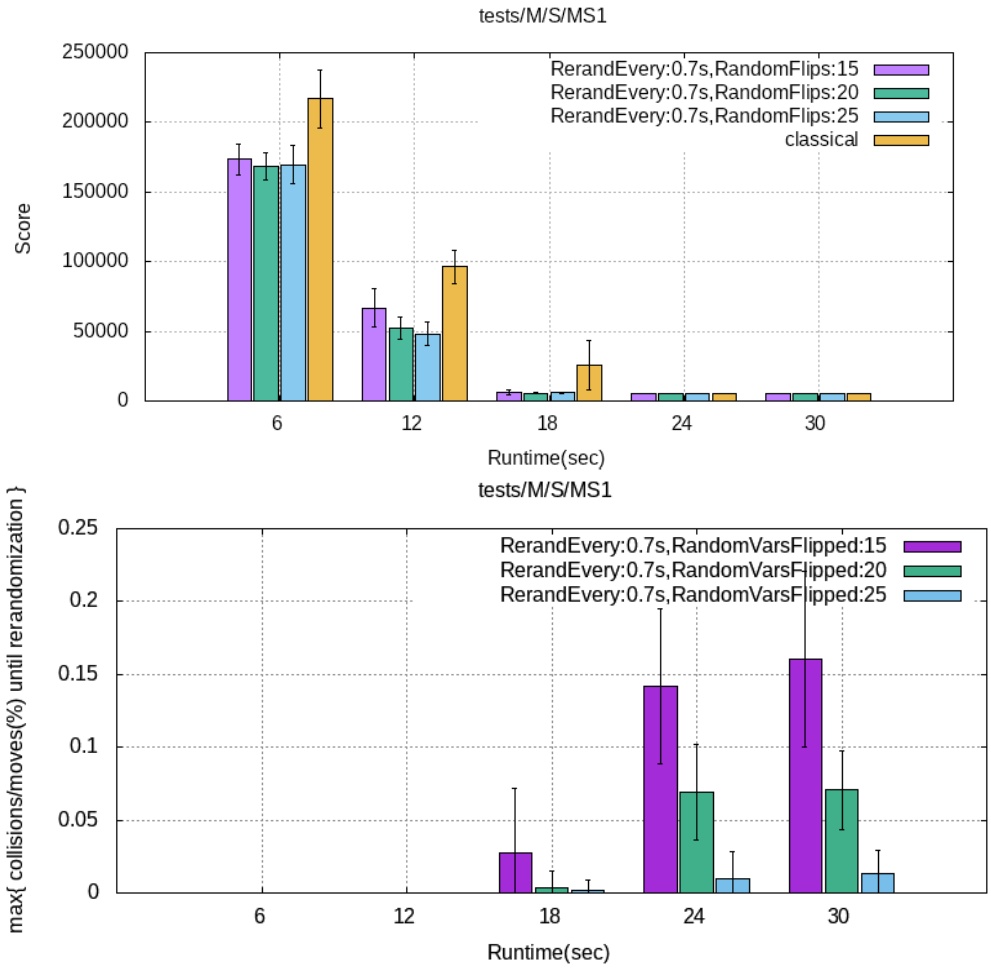

Figure 10: Warehouse Location: MS1. Quality (left) and Collisions (right).

6 Conclusion

This paper revises Ofelimos, a provably-secure PoUW blockchain protocol, proposes a new algorithmic core for its Proof-of-Useful-Work mechanism, and demonstrates the value of the resulting optimization framework in the context of two realistic, large scale combinatorial optimization problems that can deliver economic value beyond security guarantees for the blockchain. The paper discusses solver and model properties desirable to marry FRLS to Ofelimos, opening the space for using other combinatorial optimization techniques.

References

[1] K. Al-Sultan and M. Al-Fawzan. A tabu search approach to the uncapacitated facility location problem. *Annals of Operations Research*, 86:91–103, 1999.

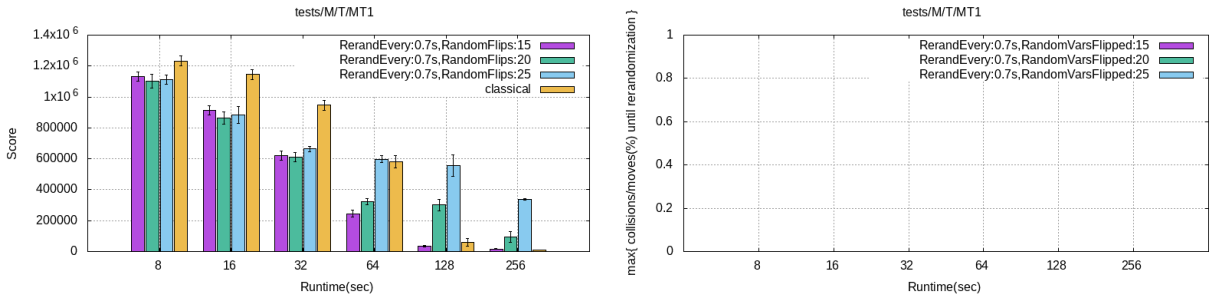

Figure 11: Warehouse Location: MT1. Solution Quality (left) and Collisions (right).

[2] M. Alves and M. Almeida. Simulated annealing algorithm for simple plant location problems. *Rev. Invest.*, 12, 1992.

[3] A. Baldominos and Y. Saez. Coin. ai: A proof-of-useful-work scheme for blockchain-based distributed deep learning. *Entropy*, 21(8):723, 2019.

[4] M. Ball, A. Rosen, M. Sabin, and P. N. Vasudevan. Proofs of work from worst-case assumptions. In *Advances in Cryptology – CRYPTO 2018*, pages 789–819. Springer, 2018.

[5] P. Baptiste, C. Le Pape, and W. Nuijten. *Constraint-Based Scheduling*. Kluwer, 2001.

[6] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza. Succinct non-interactive zero knowledge for a von neumann architecture. `https://eprint.iacr.org/2013/879`, 2013.

[7] T. Benoist, B. Estellon, F. Gardi, R. Megel, and K. Nouioua. LocalSolver 1.x: A black-box local-search solver for 0-1 programming. *4OR*, 9:299–316, Sept. 2011.

[8] M. Bofill, J. Coll, J. Suy, and M. Villaret. SMT encodings for Resource-Constrained Project Scheduling Problems. *Computers & Industrial Engineering*, 149:106777, Nov. 2020.

[9] CBECI. Cambridge bitcoin electricity consuption index. `https://ccaf.io/cbnsi/cbeci`, 2024.

[10] A. Cesta, A. Oddi, and S. F. Smith. An iterative sampling procedure for resource constrained project scheduling with time windows. In *IJCAI*, pages 1022–1033, 1999.

[11] A. Cesta, A. Oddi, and S. F. Smith. Iterative flattening: A scalable method for solving multi-capacity scheduling problems. In *AAAI/IAAI*, pages 742–747, 2000.

[12] C. Chenli, B. Li, and T. Jung. Dlchain: Blockchain with deep learning as proof-of-useful-work. In *Services–SERVICES 2020: 16th World Congress, SCF 2020*, pages 43–60. Springer, 2020.

[13] C. Chenli, B. Li, Y. Shi, and T. Jung. Energy-recycling blockchain with proof-of-deep-learning. pages 19–23, 05 2019.

[14] K. P. Choi, E. H. H. Kam, X. T. Tong, and W. K. Wong. Appropriate noise addition to metaheuristic algorithms can enhance their performance. *Scientific Reports*, 13(1):5291, Mar. 2023.

[15] P. Codognet, D. Munera, D. Diaz, and S. Abreu. *Parallel Local Search*, pages 381–417. Springer, Cham, 2018.

[16] A. Coventry. Nooshare: A decentralized ledger of shared computational resources. http://web.mit.edu/alex_c/www/nooshare.pdf, 2012.

[17] D. Erlenkotter. A Dual-Based Procedure for Uncapacitated Facility Location: General Solution Procedures and Computational Experience. *Operations Research*, 26:992–1009, 1978.

[18] M. Fitzi, A. Kiayias, G. Panagiotakos, and A. Russell. Ofelimos: Combinatorial optimization via proof-of-useful-work: A provably secure blockchain protocol. Extended version on IACR ePrint https://eprint.iacr.org/2021/1379, 2021.

[19] M. Fitzi, A. Kiayias, G. Panagiotakos, and A. Russell. Ofelimos: Combinatorial optimization via proof-of-useful-work: A provably secure blockchain protocol. In *Advances in Cryptology–CRYPTO 2022*. Springer, 2022.

[20] L. Gao and E. Robinson. Uncapacitated Facility Location: General Solution Procedures and Computational Experience. *European Journal of Operations Research*, 76:410–427, 1994.

[21] Gapcoin. https://gapcoin.org/, 2014.

[22] M. Ghallab, D. Nau, and P. Traverso. *Automated Planning: theory and practice*. Elsevier, 2004.

[23] L. Groleaz, S. Ndiaye, and C. Solnon. ACO with automatic parameter selection for a scheduling problem with a group cumulative constraint | Proceedings of the 2020 Genetic and Evolutionary Computation Conference. 2020.

[24] P. Hansen and N. Mladenovic. An introduction to variable neighborhood search. In *Meta-heuristics, Advances and Trends in Local Search Paradigms for Optimization*. Kluwer, 1998.

[25] S. Hartmann and D. Briskorn. A survey of variants and extensions of the resource-constrained project scheduling problem. *European Journal of Operational Research*, 207(1):1–14, Nov. 2010.

[26] S. Hartmann and D. Briskorn. An updated survey of variants and extensions of the resource-constrained project scheduling problem. *European Journal of Operational Research*, 297(1):1–14, Feb. 2022.

[27] P. V. Hentenryck and L. Michel. The Objective-CP Optimization System. In *Principles and Practice of Constraint Programming - CP 2013. Proceedings*. Springer, 2013.

[28] H. H. Hoos and T. Stützle. Satlib: An online resource for research on sat. *Sat*, 2000:283–292, 2000.

[29] A. Kattis and J. Bonneau. Proof of necessary work: Succinct state verification with fairness guarantees. IACR ePrint https://eprint.iacr.org/2020/190, 2020.

[30] S. King. Primecoin: Cryptocurrency with prime number proof-of-work, 2013.

[31] J. Kotary, F. Fioretto, and P. V. Hentenryck. Fast Approximations for Job Shop Scheduling: A Lagrangian Dual Deep Learning Method. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2022.

[32] J. Kratica, V. Filipovic, V. Sesum, and D. Tosic. Solving the uncapacitated warehouse location problem using a simple genetic algorithm. In *Proceedings of the XIV International Conference on Material handling and warehousing*, pages 3.33–3.37, 1996.

[33] J. Kratica, D. Tosic, and V. Filipovic. Solving the uncapacitated warehouse location problem by sga with add-heuristic. In *XV ECPD International Conference on Material Handling and Warehousing*, 1998.

[34] J. Kratica, D. Tosic, V. Filipovic, and I. Ljubic. Solving the Simple Plant Location Problems by Genetic Algorithm. *RAIRO Operations Research*, 35:127–142, 2001.

[35] P. Laborie and M. Ghallab. Planning with sharable resource constraints. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence*, pages 1643–1649, 1995.

[36] Y. Lan, Y. Liu, B. Li, and C. Miao. Proof of learning (pole): Empowering machine learning with consensus building on blockchains. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 16063–16066, 2021.

[37] A. Lihu, J. Du, I. Barjaktarevic, P. Gerzanics, and M. Harvilla. A proof of useful work for artificial intelligence on the blockchain. arXiv `https://arxiv.org/abs/2001.09244`, 2020.

[38] A. Liu, P. Luh, K. Sun, M. Bragin, and B. Yan. Integrating Machine Learning and Mathematical Optimization for Job Shop Scheduling, Mar. 2023.

[39] A. F. Loe and E. A. Quaglia. Conquering generals: an np-hard proof of useful work. In *Proceedings of the 1st Workshop on Cryptocurrencies and Blockchains for Distributed Systems*, pages 54–59, 2018.

[40] L. Michel and P. Van Hentenryck. A Simple Tabu Search for Warehouse Location. *European Journal of Operational Research*, 2004.

[41] L. Michel and P. Van Hentenryck. Iterative Relaxations for Iterative Flattening in Cumulative Scheduling. In *14th International Conference on Automated Planning & Scheduling*, 2004.

[42] B. Naderi, R. Ruiz, and V. Roshanaei. Mixed-Integer Programming vs. Constraint Programming for Shop Scheduling Problems: New Results and Outlook. *INFORMS Journal on Computing*, 35(4):817–843, July 2023. Publisher: INFORMS.

[43] S. Nakamoto. Bitcoin: A peer-to-peer electronic cash system. http://bitcoin.org/bitcoin.pdf, 2008.

[44] A. Oddi and S. F. Smith. Stochastic procedures for generating feasible schedules. In *Proceedings of the 14th National Conference on Artificial Intelligence (AAAI-97)*. AAAI Press / MIT Press, 1997.

[45] B. Parno, C. Gentry, J. Howell, and M. Raykova. Pinocchio: Nearly practical verifiable computation. IACR ePrint `https://eprint.iacr.org/2013/279`, 2013.

[46] K. Parsopoulos, V. Plagianakos, G. Magoulas, and M. Vrahatis. Objective function "stretching" to alleviate convergence to local minima. *Nonlinear Analysis: Theory, Methods & Applications*, 47(5):3419–3424, 2001.

[47] R. Rasconi, A. Oddi, and A. Cesta. Surveying the Versatility of Constraint-Based Large Neighborhood Search for Scheduling Problems. In *Beyond Databases, Architectures and Structures*, Communications in Computer and Information Science, pages 33–43, Cham, 2015. Springer.

[48] N. Shibata. Proof-of-search: combining blockchain consensus formation with solving optimization problems. *IEEE Access*, 7:172994–173006, 2019.

[49] X. Su, M. Larangeira, and K.-k. Tanaka. Provably secure blockchain protocols from distributed proof-of-deep-learning. In *International Conference on Network and System Security*. Springer, 2023.

[50] H. Turesson, M. Laskowski, A. Roatis, and H. M. Kim. Privacy-preserving blockchain mining: Sybil-resistance by proof-of-useful-work. *Available at SSRN 3206258*, 2019.

[51] P. Van Hentenryck and L. Michel. *Constraint-Based Local Search*. The MIT Press, Cambridge, Mass., 2005.

[52] Y. Wu, X. Wang, C. Chen, and G. Liu. Proof of learning: Towards a practical blockchain consensus mechanism using directed guiding gradients (student abstract). In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2022.

[53] H. Xiong, S. Shi, D. Ren, and J. Hu. A survey of job shop scheduling problem: The types and models. *Computers & Operations Research*, 142:105731, June 2022.

[54] Z. Yang, Y. Shi, Y. Zhou, Z. Wang, and K. Yang. Trustworthy federated learning via blockchain. *IEEE Internet of Things Journal*, 10(1):92–109, 2022.

[55] X. Zhao, W. Song, Q. Li, H. Shi, Z. Kang, and C. Zhang. A Deep Reinforcement Learning Approach for Resource-Constrained Project Scheduling. In *2022 IEEE Symposium Series on Computational Intelligence (SSCI)*, 2022.

A Ofelimos PoUW in Detail

In Bitcoin, PoW is implemented by repeated hashing: a counter inside the proposed block is increased until the hash value of the block lies below a given threshold—constituting the event of finding a block. Ofelimos essentially runs a variant of Bitcoin wherein this PoW is replaced by PoUW. However, PoUW does not exclusively consist of useful work but involves further steps to solve the following issues.

Issue 1: Shortcuts. A miner must not be able to reduce the time complexity of its useful-work computations by biasing the randomization towards easy-to-compute instances, and thus gaining an advantage in the block lotteries over miners that apply uniform randomness. Not only would this harm the security of the blockchain protocol but it might also imply that PoUW attempts with higher-than-average computation cost would never be executed.

Even worse, a dishonest party could post a "pre-analyzed" optimization problem as a client themselves, and later work on this very problem as a blockchain miner, driving their local useful-work computations towards precomputed instances, thereby reducing the required amount of online computation and solving PoUW substantially faster than honest miners.

Issue 2: Verification overhead. Since participation in the blockchain protocol is public, the protocol must be secured against adversarial behavior. To that end, all nodes in the network need to verify that the block miners performed their PoUWs correctly—similarly to verifying that the Bitcoin block hash lies below the threshold.

The ratio of verification computation in the network per executed useful-work computation must be minimized in favor of the overall "usefulness" of the PoUW blockchain.

In particular, a single attempt to find a block (the analog of one PoW hash in Bitcoin) consists of the three major steps: pre-hashing, useful work, and post-hashing.

Pre-hashing. The purpose of the *pre-hashing* step is to defend against "shortcuts." It enforces the miner to first solve a "useless" standard instance of PoW (à la Bitcoin) whose average-case complexity is at least as high as the worst-case complexity of the useful-work computation, and yielding an unpredictable `seed` that dictates the (pseudo-)randomness to be applied to the useful-work computation. As a consequence, trying to re-sample the given seed (by repeating the pre-hashing step) in hope of obtaining a new useful-work instance that is easier to compute, comes at a cost that is at least as high as computing the given instance. This incentivizes the miner to perform whatever instance they are initially served—independently of its computation complexity.

Post-hashing. The purpose of the *post-hashing* step is to minimize verification overhead. The output of the useful computation (together with the pre-hash output) is hashed, and a block is found only if this hash lies below a given threshold. This implies that, in expectation, a large number of useful-work updates must be computed until a block is found. Instead of publishing all useful-work updates computed during this course, only the best such update is published for further consideration by the distributed computation. Thus only a small number of good updates need to be publicly verified—compared to the full amount of useful work computed in the system.

As an additional measure to minimize verification in Ofelimos, but not made explicit in this paper, the block producer proves that they computed their PoUW correctly (as prescribed by the algorithm and the prescribed seed) by use of a succinct non-interactive argument (SNARG). A SNARG system allows to compute a non-interactive proof of correctness that can be verified (essentially) in constant time. Thus,

verification by the many nodes in the system is reduced to constant time against a reasonably small overhead by the block producer.

The PoUW function. Consider Fig. 12 for a more detailed picture. The given pseudo-code can be thought of as the equivalent of a single PoW evaluation against the threshold in Bitcoin.

```
1  PoUW(block, problem,algSt,localSt):
2  ⟨blockFound,cnt,algSt,localSt⟩ {
3    cnt = 0;
4    do { //pre-hash
5      cnt = cnt + 1;
6      preH = Hash(block,problem,algSt,cnt);
7    } until ( preH < T1 );
8    seed = Hash(preH); //(pseudo)random
9    ⟨algSt',localSt'⟩ = usefulWork(problem,algSt,localSt,seed);
10   postH = Hash(preH, algSt');
11   blockFound = postH < T2; // post-hash
12   return ⟨blockFound,cnt,algSt',localSt'⟩;
13 }
```

Figure 12: The pseudo-code for a single PoUW attempt.

The PoUW function is called from the miner environment that is responsible for observing the blockchain. Note that, once a block is found, the miner's computational progress in the useful computation is included in the block in form of a state update—so other miners can update their local views, and extend on other miners' updates. To this effect, the "public" algorithmic state of the useful computation is fully implied by the state of the blockchain.

PoUW Arguments. PoUW gets passed the new `block` the miner intends to append to the blockchain (and, implicitly, the current state of the full blockchain, as the block points to the last block of the current chain). The description `problem` identifies the instance of the useful-work problem the miner is working on. Additionally, the state `algSt` of the distributed computation is explicitly passed—as remotely updated by new published blocks as well as locally updated by this function.[8] Also, a *local* state `localSt` holding auxiliary data-structures needed by the `usefulWork` routine is held in, and passed from, the calling environment. The local state is helpful to avoid repeatedly setting up auxiliary structures at each invocation of `usefulWork`. In the case of Constraint-Based Local Search, these would be the data-structures needed to incrementally maintain the values of functionally dependent variables as well as the violations of constraints appearing in the declarative model.

PoUW Computation. Lines 1-7 compute the pre-hash of the block, problem description and the current algorithmic state. It starts by setting a counter `cnt` to 0 within the block and increases the counter until the pre-hash lies below `T1`. Line 8 fixes the seed of the pseudo-random stream to be used by the miner from the prehash itself. Line 9 invokes the `usefulWork` computation for the instance in `problem` and both the current `algSt` and `localSt`. The call returns a new version of both the algorithmic and local state. Line 10 hashes the `algSt` together with the `preH`. This resulting `postH` is then compared against threshold `T2`

[8]This is not necessarily the full state of the distributed computation but possibly only the state of one of multiple concurrent threads.

to determined whether a block was found. Line 12 finally outputs the result of `PoUW` in the form of a tuple reporting whether or not a block was found, the counter `cnt` and the updated useful-work states `algSt'` and `localSt'`.

Calling environment, state updates, and block structure. Recall that, among the many algorithmic state updates written back by PoUW, a block is found only once in a while; and the calling environment is responsible of memorizing the best of these updates (by means of the objective function) for the later publication in a block.

During the course of mining a block, two state updates are thus of special interest, the *best update* to contribute progress to the distributed useful computation, and the *winning update* that lead to finding a new block by yielding a low `postH`. And, as a consequence, both updates have to be stored in the block, and need to be verified by the public.

A state update needs to describe the `algSt` (that was passed to `usefulWork`) together with the resulting `algSt'`. This information is sufficient for verification since, given the seed that is "dictated" by the blockchain (and extractable from the block hash), the state update is deterministic. Typically, the size of `algSt` is small compared to total block capacity—e.g., in CBLS, listing the set of all decision variables is sufficient.

In conclusion, once a block has been found (`blockFound`), the caller needs to write back the "winning" counter `cnt` into the block and *attach* the best and the winning state updates (whereas this attached data is not part of the hashed block). A block must thus contain the following data: link to previous block, ledger transactions, `cnt`, winning update, and, best update.

B More WalkSAT Experiments

We run the different WalkSAT variants on the following benchmarks found in the SATLIB [28]: SAT-encoded bounded model checking (instances `bmc-ibm-{1,2,3}.cnf` and `bmc-galileo-{8,9}.cnf`) shown in Figs. 14 and 13. In all tests it is obvious that DPLS-WalkSAT converges to a good solution in a significantly slower manner than either classical WalkSAT or FRLS-WalkSAT. On the other hand, the performance of FRLS-WalkSAT is close to that of classical WalkSAT. Thus, the experimental results confirm our intuition.

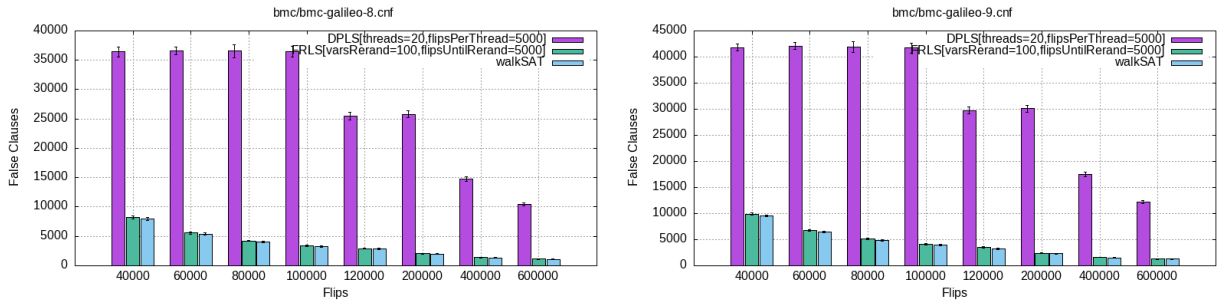

Figure 13: Walksat – Bounded Model Checking, galileo instances.

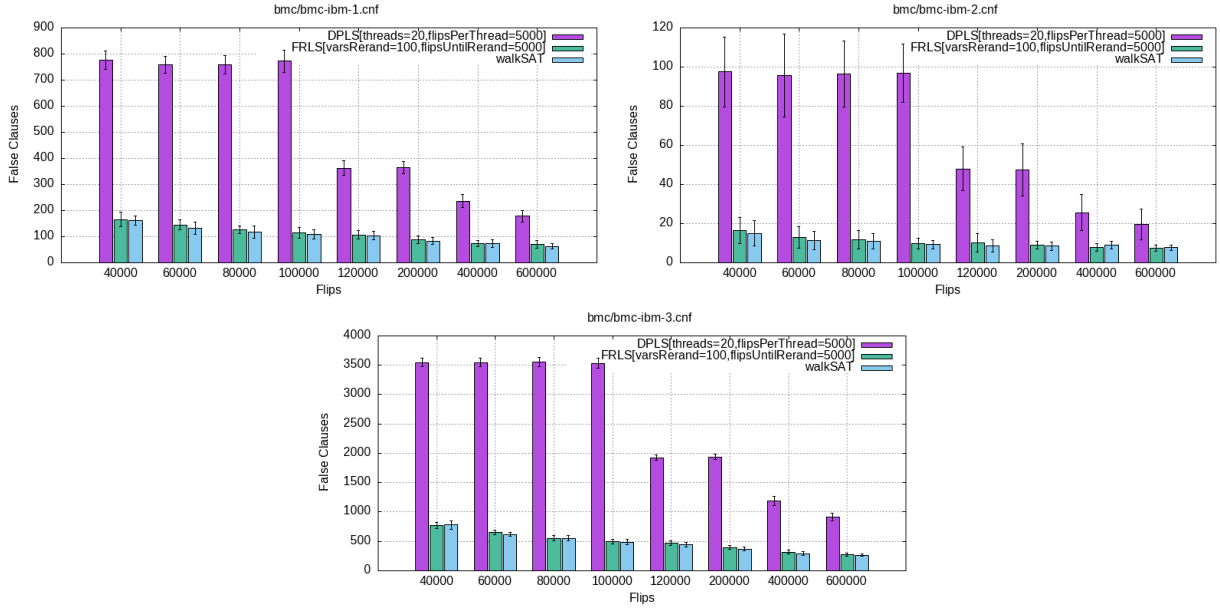

Figure 14: Walksat – Bounded Model Checking.